CM32024 Bayesian Machine Learning CW2 Part B Report

Ryan Kansara
rk955@bath.ac.uk

Oliver Gibbard
jd2452@bath.ac.uk

Deniz Taskin
dbt26@bath.ac.uk

Jayden Wong
cyw89@bath.ac.uk

1 Surrogate GP for Preference Data

The primary change made from part A is the likelihood. In Part A, the regression expected log-likelihood had a closed-form Gaussian expression - the familiar $-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y-\mu)^2}{2\sigma^2}$ plus a trace term. Part B replaces this with a Bradley-Terry log-likelihood over pairs:

$$y \cdot \log\sigma(f_i - f_j) \;+\; (1-y)\cdot\log\sigma(f_j - f_i),$$

implemented using log-softplus for numerical stability. This is the same MC reparameterisation trick from Part A's classification task, just applied to the difference $f(\mathbf{x}_i) - f(\mathbf{x}_j)$ rather than to individual function values.

For joint sampling, we need the posterior over $f(\mathbf{x}_i) - f(\mathbf{x}_j)$, which is itself Gaussian with mean $\mu_i - \mu_j$ and variance $\Sigma_{ii} + \Sigma_{jj} - 2\Sigma_{ij}$. Getting this right means computing the cross-covariance Σ_{ij} - the $k_{ij} - \boldsymbol{\alpha}_i \cdot \boldsymbol{\alpha}_j + \mathbf{B}_i \cdot \mathbf{B}_j$ term. This is derived from the spec:

$$f(\mathbf{x}_i) - f(\mathbf{x}_j) \;\sim\; \mathcal{N}\big(\mu_i - \mu_j,\; \Sigma_{ii} + \Sigma_{jj} - 2\Sigma_{ij}\big).$$

We only need the diagonal elements of these covariance blocks. This keeps everything computationally cheap.

For effiency, rather than evaluating the kernel separately for X_i_batch and X_j_batch, we stack them into a single $(2B, 2)$ array, solve the triangular system once, and split the result afterwards. This avoids redundant kernel evaluations.

```python
def svgp_ell_preference(params, X_i_batch, X_j_batch, y_ij_batch, N_total, rng_key, n_samples=20):
    ls  = jnp.exp(params["log_lengthscale"])
    var = jnp.exp(params["log_variance"])
    Z   = params["inducing_inputs"]   # (M, 2)
    m   = params["variational_mean"]  # (M,)
    M   = Z.shape[0]
    L_v  = build_var_chol(params["var_chol_log_diag"], params["var_chol_lower"], M)
    K_zz = rbf_kernel(Z, Z, ls, var)
    L_zz = stable_cholesky(K_zz)
    B_size = X_i_batch.shape[0]

    # Stack both halves to run the kernel and solve once
    X_pairs   = jnp.concatenate([X_i_batch, X_j_batch], axis=0)              # (2B, 2)
    K_z_pairs = rbf_kernel(Z, X_pairs, ls, var)                               # (M, 2B)
    alpha_all = jsp.linalg.solve_triangular(L_zz, K_z_pairs, lower=True)      # (M, 2B)

    alpha_i = alpha_all[:, :B_size]    # (M, B)
    alpha_j = alpha_all[:, B_size:]    # (M, B)
    B_i     = L_v.T @ alpha_i          # (M, B)
    B_j     = L_v.T @ alpha_j          # (M, B)

    # Posterior means for each side
    mu_i    = alpha_i.T @ m            # (B,)
    mu_j    = alpha_j.T @ m            # (B,)
    mu_diff = mu_i - mu_j              # (B,)

    # Posterior variances and cross-covariance — only diagonal elements needed
    k_ii = rbf_kernel(X_i_batch, X_i_batch, ls, var, diag=True)              # (B,)
    k_jj = rbf_kernel(X_j_batch, X_j_batch, ls, var, diag=True)              # (B,)
    k_ij = rbf_kernel(X_i_batch, X_j_batch, ls, var, diag=True)              # (B,)

    Sigma_ii = k_ii - jnp.sum(alpha_i ** 2, axis=0) + jnp.sum(B_i ** 2, axis=0)       # (B,)
    Sigma_jj = k_jj - jnp.sum(alpha_j ** 2, axis=0) + jnp.sum(B_j ** 2, axis=0)       # (B,)
    Sigma_ij = k_ij - jnp.sum(alpha_i * alpha_j, axis=0) + jnp.sum(B_i * B_j, axis=0) # (B,)

    var_diff = jnp.clip(Sigma_ii + Sigma_jj - 2.0 * Sigma_ij, 1e-6)          # (B,)
    std_diff = jnp.sqrt(var_diff)                                             # (B,)

    # Reparametrised Monte Carlo samples
    rng_key, subkey = jax.random.split(rng_key)
    eps     = jax.random.normal(subkey, shape=(n_samples, B_size))            # (S, B)
    f_diff = mu_diff[None, :] + std_diff[None, :] * eps                       # (S, B)

    # Log-Bernoulli via log-sigmoid (numerically stable)
    y       = y_ij_batch[None, :].astype(jnp.float32)                         # (1, B)
    log_lik = y * (-jax.nn.softplus(-f_diff)) + \
              (1.0 - y) * (-jax.nn.softplus(f_diff))                          # (S, B)

    ell = (N_total / B_size) * jnp.sum(jnp.mean(log_lik, axis=0))
    return ell, rng_key
```

Figure 1: The `svgp_ell_preference` function. The top section stacks `X_i_batch` and `X_j_batch` to solve the triangular system once (kernel stacking trick). The middle section computes the cross-covariance Σ_{ij} and the variance of the difference $f(\mathbf{x}_i) - f(\mathbf{x}_j)$. The bottom section applies the reparametrisation trick and evaluates the Bradley-Terry log-likelihood via log-softplus.

From a structural standpoint, there are some changes. For example, there is no noise variance parameter. Part A had `log_noise_var` because it had noisy function observations; Part B drops it because preferences are noiseless observations of a latent comparison - the noise enters through the sigmoid, not as additive Gaussian noise.

Additionally, the inducing inputs are initialised differently. Part A used linspace since the input was 1D, but in 2D there is no natural ordering and we don't have training locations to cluster over (we only have pairs), so we used random uniform initialisation over $[-3, 3]^2$. Similarly, the variational

mean is initialised at zero rather than by interpolating training targets, since in the preference setting there are no direct function observations to interpolate.

2 Thompson Sampling

Thompson Sampling is used for the preference setting, where each query requires two candidate points instead of one. To do Thompson Sampling, we draw two independent samples from the GP posterior using the marginal approximation - sample $f \sim \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$ at a random candidate set - and then take the argmax of each sample. Those two argmaxes become the duel candidates. Using the marginal (diagonal) variance rather than the full joint is a deliberate approximation: computing the full joint would require an expensive $(n_{\text{candidates}} \times n_{\text{candidates}})$ Cholesky, and since we are just finding argmaxes rather than computing integrals over the posterior, a diagonal approximation is sufficient.

```python
#Select two candidate points via Thompson sampling. each candidate is argmax of an independent sample from GP posterior
def thompson_sampling(params, rng_key, n_candidates=2000):
    rng_key, k_grid, k_eps1, k_eps2 = jax.random.split(rng_key, 4)

    # Random candidates uniformly over the input domain
    candidates = jax.random.uniform(k_grid, shape=(n_candidates, 2),
                                    minval=-3.0, maxval=3.0)

    # Posterior mean and marginal variance at all candidates
    mu, var = svgp_predict(params, candidates, full_cov=False)  # both (n_candidates,)
    std = jnp.sqrt(jnp.clip(var, 1e-6))

    # Sample 1
    eps1   = jax.random.normal(k_eps1, shape=(n_candidates,))
    f1     = mu + std * eps1
    cand1  = candidates[jnp.argmax(f1)]                        # (2,)

    # Sample 2 — independent eps, same mu/std (marginal approximation)
    eps2   = jax.random.normal(k_eps2, shape=(n_candidates,))
    f2     = mu + std * eps2
    cand2  = candidates[jnp.argmax(f2)]                        # (2,)

    return cand1, cand2, rng_key
```

Figure 2: The `thompson_sampling` function, drawing two independent marginal posterior samples and returning their argmaxes as duel candidates.

3 Bayesian Optimisation Loop

The loop starts with 5 random initial points and 25 preference pairs, using `seed=7` (changed from the default 42 to ensure the initial data are not sitting on the peak). The initial points end up far from the true optimum at $[1.2, -1.0]$, which is what we want - the whole point is to watch the optimiser find it.

From there, the loop runs for 50 rounds. Each round draws two candidates via Thompson Sampling, collects 25 noisy preference samples for that duel, appends them to the dataset, and warm-start retrains the model for 100 epochs at a learning rate of 0.005 with batch size 64. Warm-starting is significantly cheaper as it means we can continue from the previous solution rather than reinitialising from scratch every round.

```python
print("Starting Bayesian Optimisation loop (50 rounds)...")
for round_idx in range(50):
    rng_key, ts_key = jax.random.split(rng_key)
    cand1, cand2, _ = thompson_sampling(params, ts_key, n_candidates=2000)

    rng_key, sp_key = jax.random.split(rng_key)
    new_Xi, new_Xj, new_yij = sample_preferences(sp_key, cand1, cand2,
                                                 preference_samples_per_location=25)
    X_i  = jnp.concatenate([X_i,  new_Xi])
    X_j  = jnp.concatenate([X_j,  new_Xj])
    y_ij = jnp.concatenate([y_ij, new_yij])

    params, rng_key = train_preference_gp(
        params, X_i, X_j, y_ij, rng_key, num_epochs=100, lr=0.005, batch_size=64
    )

    print(f"  Round {round_idx + 1}/50  |  total pairs: {y_ij.shape[0]}  |  "
          f"cand1=[{float(cand1[0]):.2f}, {float(cand1[1]):.2f}]  "
          f"cand2=[{float(cand2[0]):.2f}, {float(cand2[1]):.2f}]")

print("BO loop complete.")
```

Figure 3: The main Bayesian optimisation loop: each iteration performs Thompson Sampling, gathers 25 preference samples for the chosen duel, appends data, and warm-start retrains the surrogate.

3.1 Hyperparameters

Parameter	Initial training	BO loop
`num_inducing`	15	15
`batch_size`	32	64
`num_epochs`	300	100
`learning_rate`	0.01	0.005
`n_samples` (MC ELL)	20	20
`n_candidates` (TS)	-	2000
`pairs per round`	-	25
`seed` (initial data)	7	
`rounds`	50	

Table 1: Hyperparameters used for the initial surrogate training and the BO loop.

3.2 Initial Surrogate Fit

The initial surrogate (trained on the seed data only) is smooth but poorly informed - it does not identify the peak at all. The two plots below show the surrogate model's mean prediction and the true latent function, with the collected data locations overlaid.

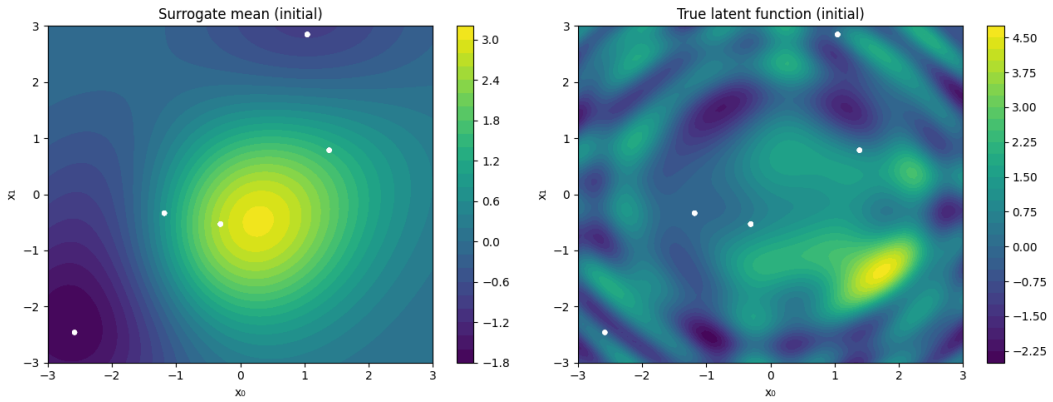

Figure 4: Initial surrogate mean (left) and true latent function (right), with initial data locations overlaid. The model has not yet identified the peak.

3.3 Final Surrogate Fit (After 50 Rounds)

By the end, the surrogate correctly locates the peak near $[1.2, -1.0]$, and the queried points cluster tightly around it. The colour coding makes this easy to see: later queries (warmer colours) concentrate around the optimum, demonstrating the BO loop converging as expected.

The surrogate peak sits around ~ 4.2, close to the true peak of ~ 4.5. Any residual offset is expected: the Bradley-Terry likelihood only identifies the latent function up to positive scaling, so what matters is the *location* of the optimum, which is correct.

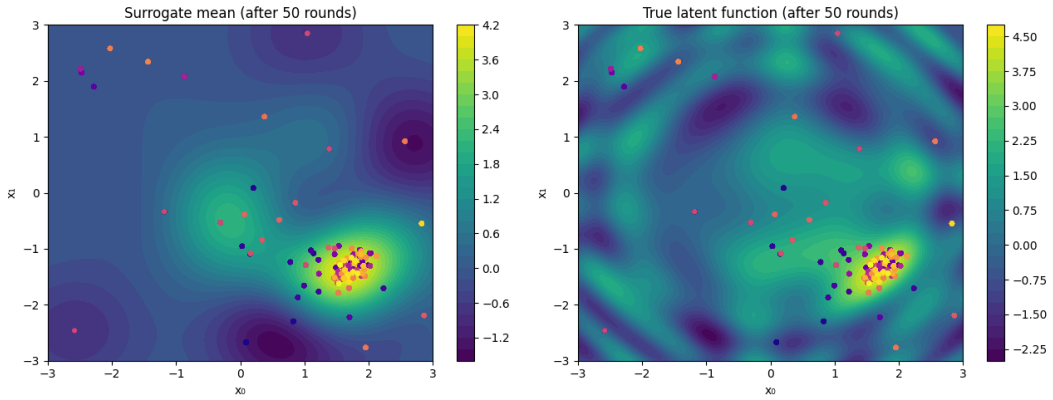

Figure 5: Final surrogate mean (left) and true latent function (right) after 50 BO rounds. Queried points are colour-coded by iteration (warmer = later), showing convergence toward the global optimum near $[1.2, -1.0]$.

4 Written Questions (CM32024)

1. Why does it make sense to use a sparse GP in this setting, rather than just a full-dataset variational inference approximation?

The dataset grows every round - it starts at 25 pairs and ends at 1,275. A full GP would need $\mathcal{O}(N^3)$ computation per training step, which gets expensive fast. The sparse approximation keeps the cost at $\mathcal{O}(NM^2)$, where $M = 15$ is fixed regardless of how large the dataset gets, i.e. we have linear growth instead of polynomial growth.

2. Why can we not (at least straightforwardly) use the acquisition functions we saw in the lectures for this task?

The acquisition functions from the lectures (EI, UCB, PI) each select a single point at which to query $f(\mathbf{x})$. To use them in the preference setting, we would need to somehow produce a second candidate to form a duel - for example, by taking the acquisition maximiser as one candidate and choosing a second point via some ad hoc rule (e.g. the runner-up, or a random location). But this is problematic: these acquisition functions were derived assuming we receive a direct function value back, not a relative comparison. We would need to redesign the acquisition function from scratch to account for the information gained from a pairwise preference rather than a point observation. Thompson Sampling sidesteps this entirely - it naturally produces two candidates by taking the argmax of two independent posterior samples, with no modification needed for the preference setting.

3. What effect does the temperature parameter β have on the preference data we generate?

When $\beta \ll 1$, the sigmoid becomes a near-step function: almost every comparison produces a deterministic preference, regardless of whether the true quality difference is large or small. This means the model can no longer distinguish between pairs that are close in quality and pairs that are far apart - both yield the same hard label. The soft probabilistic signal that helps the GP calibrate the shape of the latent function is lost, and in practice the near-zero gradients through the saturated sigmoid also make training numerically difficult.
When $\beta \gg 1$, the sigmoid flattens toward 0.5: preferences approach random coin flips regardless of the true difference, and each comparison carries very little information.
When $\beta = 1$ (the default in the Bradley–Terry model), the sigmoid has moderate slope: the better option is preferred most of the time, but with enough stochasticity to reflect true uncertainty. Each comparison provides a meaningful but not overwhelming signal about relative quality.

4. If $\beta \gg 1$, what adjustment would we need to make to our Bayesian optimisation loop to ensure we still get a well-fit surrogate model?

When $\beta \gg 1$, the data-generating process uses $\sigma\big(\frac{f(\mathbf{x}_i)-f(\mathbf{x}_j)}{\beta}\big)$, but our surrogate's likelihood assumes $\sigma\big(f(\mathbf{x}_i) - f(\mathbf{x}_j)\big)$ with $\beta = 1$. To fix this mismatch, we need to incorporate β into the expected log-likelihood computation - specifically, replace the log-likelihood term

$$y \cdot \log \sigma(f_i - f_j) + (1 - y) \cdot \log \sigma(f_j - f_i)$$

with

$$y \cdot \log \sigma\Big(\frac{f_i - f_j}{\beta}\Big) + (1 - y) \cdot \log \sigma\Big(\frac{f_j - f_i}{\beta}\Big).$$

Without this, the surrogate would interpret the near-random preferences as evidence that all points have similar quality, leading to an overly flat posterior. We may also need to increase the number of preference samples per pair, since each individual comparison carries less information when β is large.